CMPT 295

Unit - Machine-Level Programming

Lecture 16 – Assembly language –
Function Call and Stack

Compiler can produce different instruction combinations when assembling the same C code.

Last Lecture

- In x86-64 assembly, there are *no conditional statements*, however, we can alter the execution flow of a program by using …
  - **`cmp*`** instructions (compare)
  - **`jX`** instructions (jump)
  - **`cmovX`** instructions (conditional move)
- In x86-64 assembly, there are *no iterative statements*, however, we can alter the execution flow of a program by using …
  - **`cmp*`** instructions
  - **`jX`** instructions (jump)
- Microprocessor uses the ***condition codes*** to decide whether a …
  - **`jX`** instruction (conditional jump) is to be exectued or a
  - **`cmovX`** instruction (conditional move) is to be exectued

Instructions such as arithmetic and logical instructions, **`cmp*`** and **`test*`** set condition codes

Today's Lecture

- Introduction
  - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
  - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
  - Overview of Function Call
  - Memory Layout and Stack - x86-64 instructions and registers
  - Passing control
  - Passing data – Calling Conventions
  - Managing local data
  - Recursion
- Array
- Buffer Overflow
- Floating-point operations

associated with the management of the Stack

What happens when a function (*caller*) calls another function (*callee*) in C?

1. **Control** (execution flow)
   - **Control** (execution flow) is passed to the beginning of the code in ***callee*** function
   - And once ***callee*** function has executed, **control** is passed back to ***caller*** function, at the code that comes right after the call to ***callee*** function
2. **Data**
   - **Data** is passed to ***callee*** function via ***function parameter(s)***
   - And once ***callee*** has executed, **data (result)** is passed back to ***caller*** function via ***return value***
3. **Memory**
   - **Memory** is allocated when ***callee*** function starts executing
   - **Memory** is deallocated when ***callee*** function stops executing

```
void who(…) {
  int sum = 0;
  ...
  y = amI(x);
  sum = x + y;
  return;
}
```

```
int amI(int i)
{
  int t = 3*i;
  int v[10];
  ...
  return v[t];
}
```

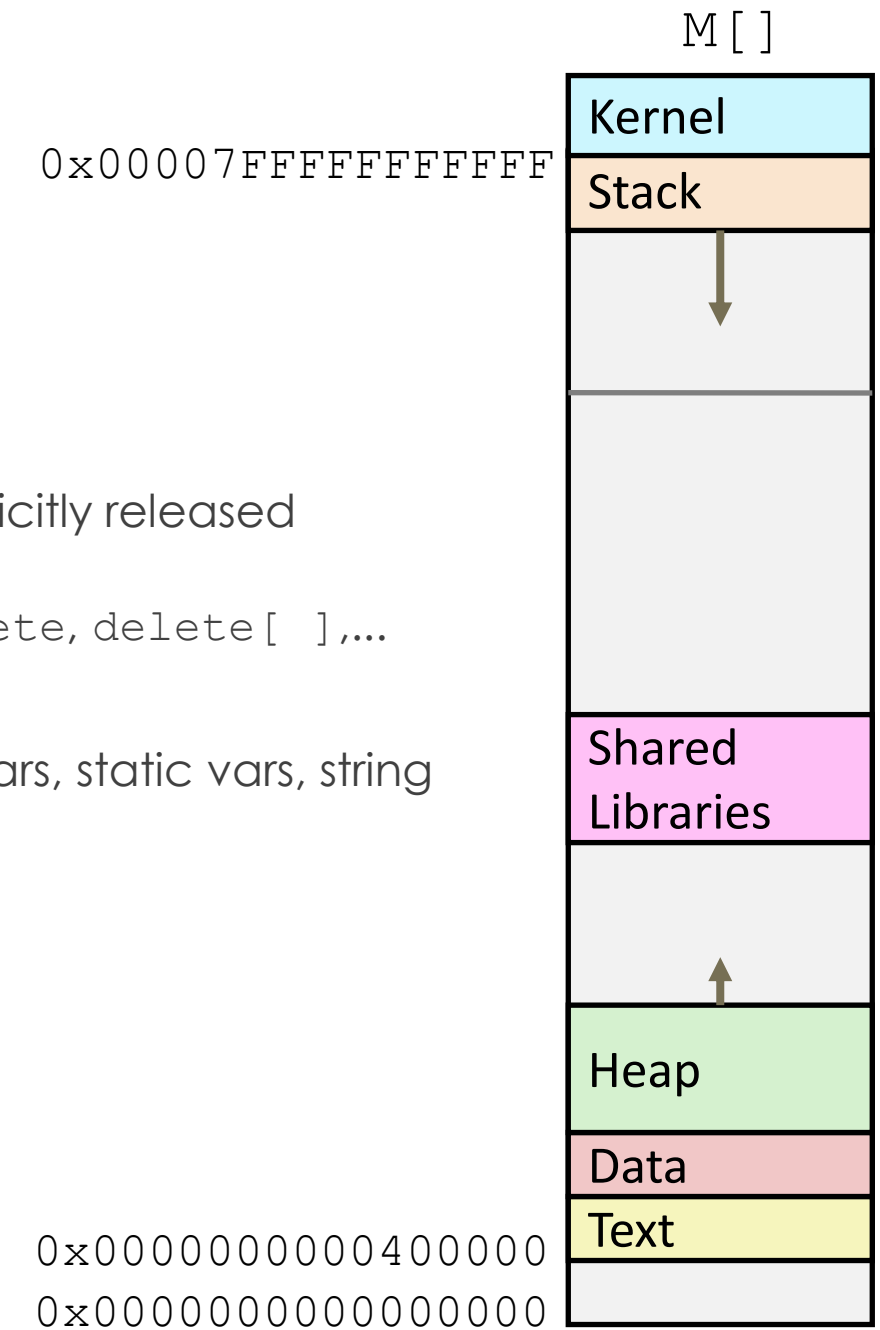
The mechanisms implementing the above are described by a set of conventions: ***function call conventions***, which are part of **Instruction Set Architecture** (ISA).

Looking into 3. Memory:

Memory Layout

segments

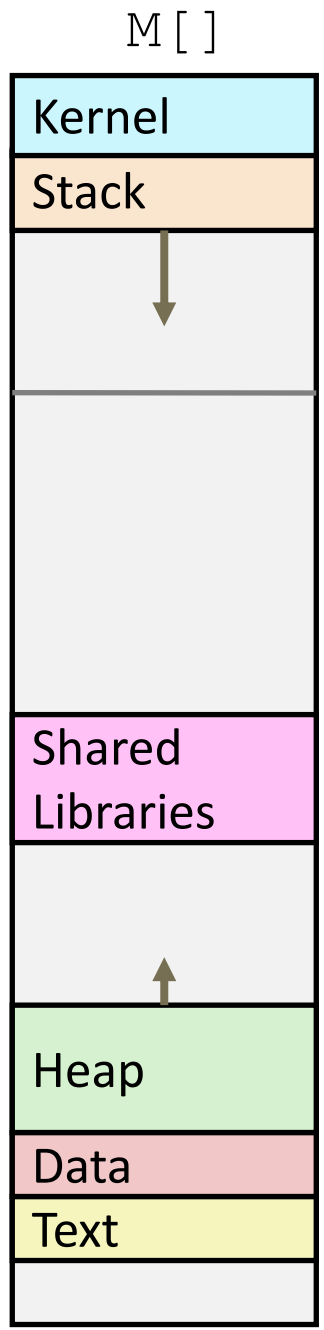
- **Stack**
  - Runtime stack, e. g., local variables
- **Heap**
  - Dynamically allocated as needed, explicitly released (freed)
  - When call `malloc()`, `free()`, `new`, `delete`, `delete[ ]`,…
- **Data**
  - Statically allocated data, e.g., global vars, static vars, string constants
- **Text**
  - Executable machine instructions
  - Read-only
- **Shared Libraries**
  - Executable machine instructions
  - Read-only



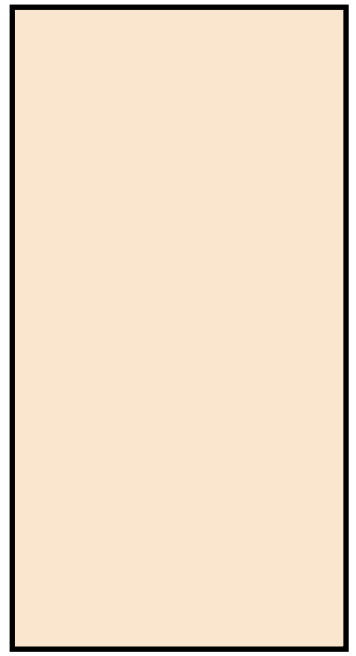
Activity: ***Memory Allocation***

Where does everything go when we invoke the loader (**`./ss`**)?

```
#include ...

char hugeArray[1 << 31]; /* 2^31 = 2GB */
int global = 0;

int useless(){ return 0; }

int main () {

  void *ptr1, *ptr2;
  int local = 0;
  ptr1 = malloc(1 << 28); /* 2^28 = 256 MB*/
  ptr2 = malloc(1 << 8);  /* 2^8  = 256 B*/

  /* Some print statements ... */
}
```



A look at **memory** being allocated/ deallocated during ***Function call***



- When a function is called (i.e., starts executing) -> ***callee***, it is allocated a stack frame

```
who(…) {
   ...
   ...
   are();
   ...
   ...
}
```

```
are(…) {
   ...
   you();
   ...
   you();
   ...
}
```

```
you(…) {
   ...
   ...
   ...
   ...
   ...
}
```

- A ***callee*** becomes a ***caller*** when it calls a function (a new ***callee***), which is also allocated a stack frame, …
- When a function (***callee***) terminates and returns, its stack frame is deallocated and its ***caller*** resumes, …
- Stack frame contains local variables and parameters

Overview of Function Call

What happens when a function (***caller***) calls another function (***callee***) in **x86-64 Assembly Language**?

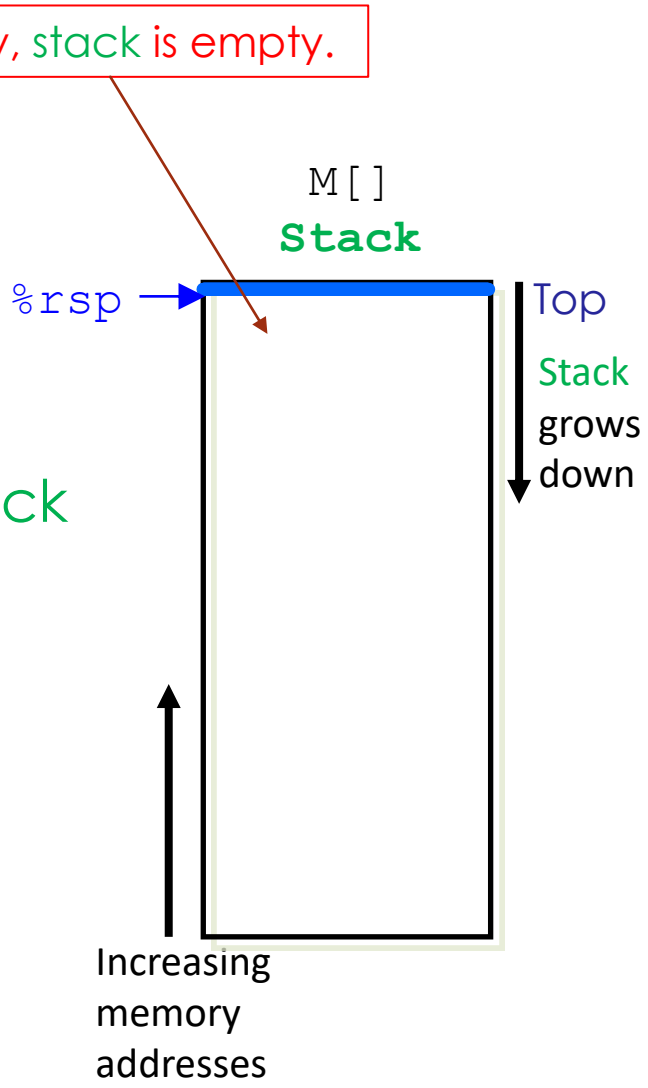
1. **Control** (execution flow)
2. **Data**
3. **Memory** -> stack memory segment

Looking into the ***function call conventions*** of **x86-64 Assembly Language**

Closer look at *stack memory segment*

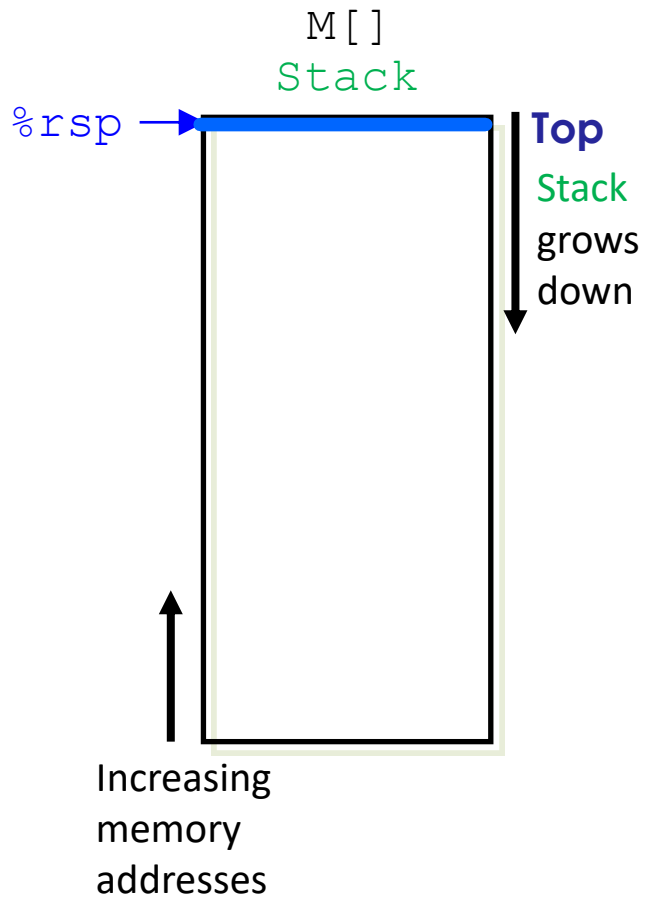
- x86-64 assembly language has stack-specific instructions and register
- `%rsp`
  - Points to address of last used byte on stack
  - Initialized to "top of stack" at startup
  - Stack grows downwards
    - Towards lower memory address
- **`pushq Src`**
- **`popq Dest`**

x86-64 stack instructions



x86-64 stack instruction: **pushq**

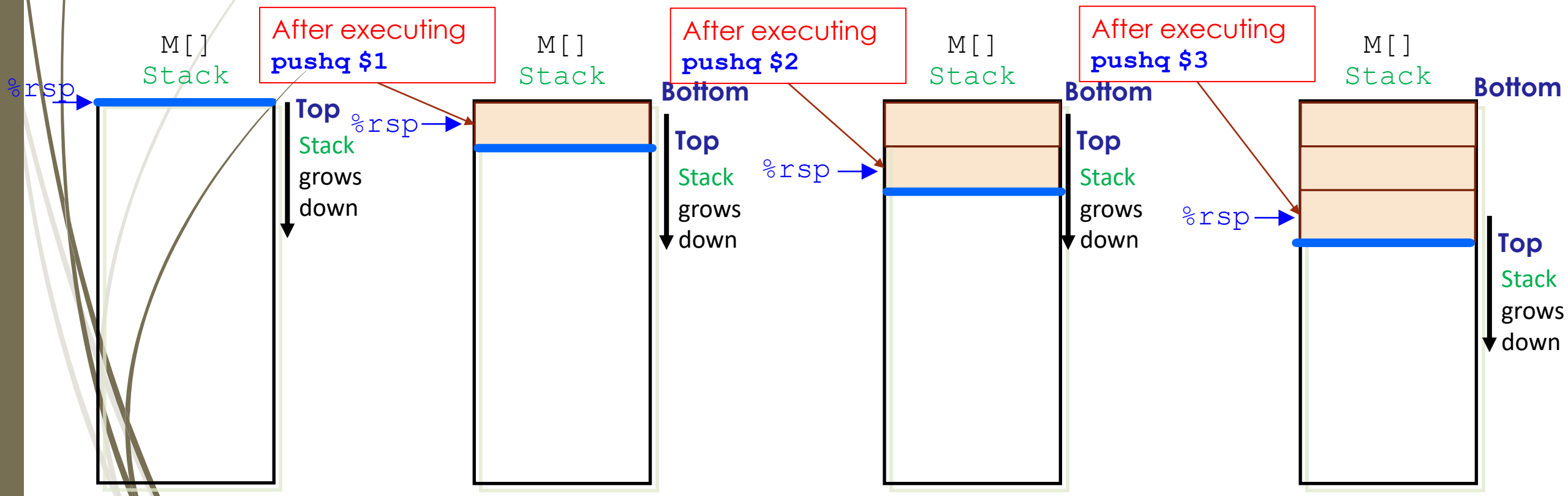
- Syntax: **pushq Src**
- **Src** can be
  - **Imm**
  - register
- What happens when **pushq** executes:
  1. Get value of operand **Src**
  2. Decrement **%rsp** by **8**
  3. Store value at memory address given in **%rsp**
- In the **pushq Src** instruction, **%rsp** is an **implicit** register



x86-64 stack instruction: **pushq**

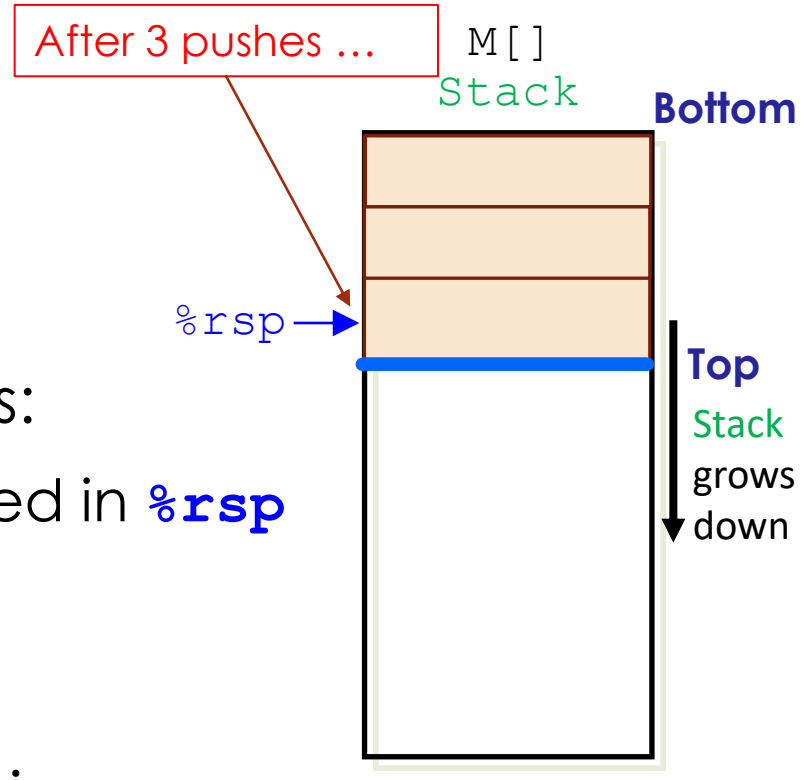
- Let execute 3 pushes:

```
pushq $1
pushq $2
pushq $3
```



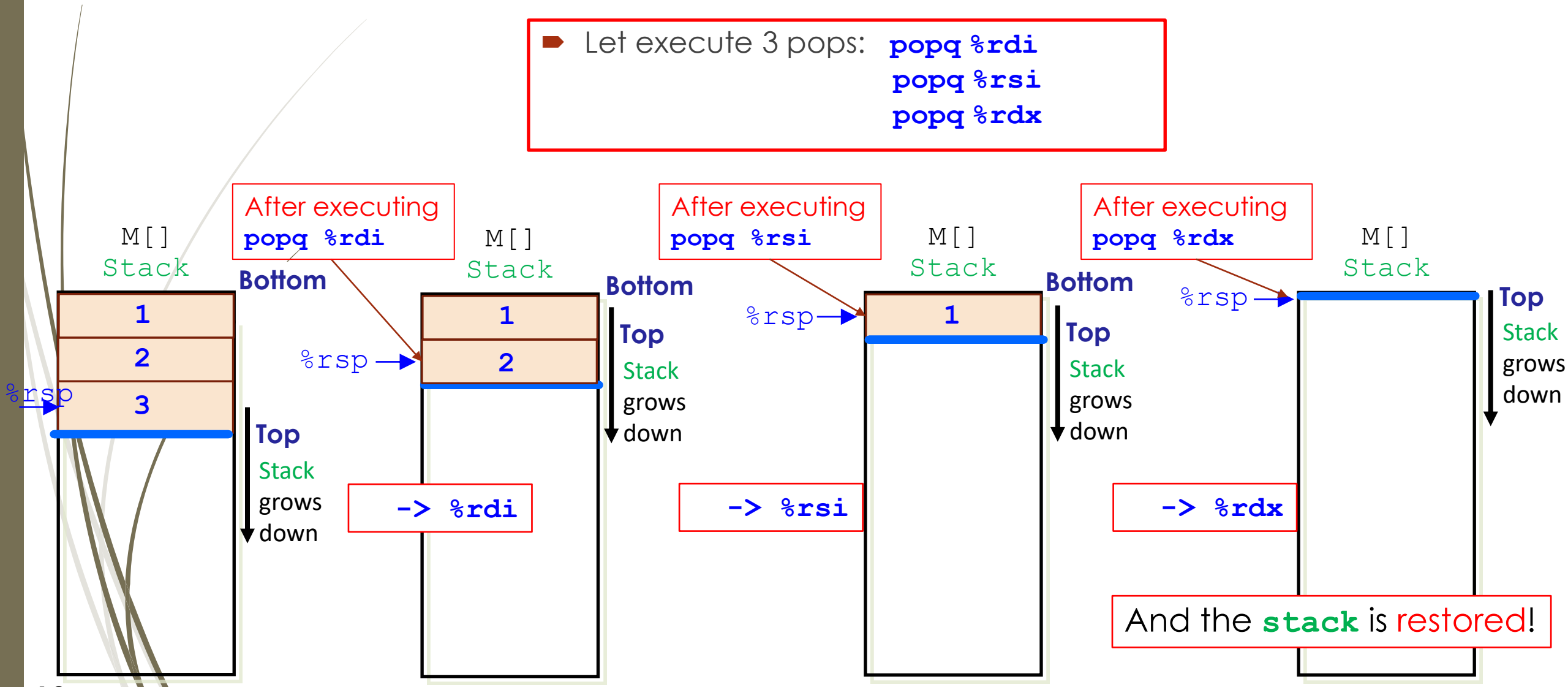
x86-64 stack instruction: **popq**

- Syntax: **popq Dest**
- **Dest** can be
  - register
- What happens when **popq** executes:
  1. Read value at memory address stored in **%rsp**
  2. Increment **%rsp** by **8**
  3. Load this value in operand **Dest**
- In the **popq Dest** instruction, **%rsp** is an **implicit** register



x86-64 stack instruction: popq

- Let execute 3 pops: `popq %rdi`
  `popq %rsi`
  `popq %rdx`



Summary

- **Function call conventions (protocol)** describing …
  1) Mechanisms for ***passing control***
  2) Mechanisms for ***passing data***
  3) Mechanisms for ***managing memory allocation***
     - **Stack frame** allocated to each executing function on **stack memory segment** and deallocated when function terminates
     - Therefore, ***function call*** has a **LIFO/FILO** behavior
- Memory layout – various segments: Stack, Heap, Data, Text / Shared Libraries
- Function call mechanisms in x86-64
  - **r**egister **s**tack **p**ointer: **`%rsp`**
  - instructions: **`pushq`** and **`popq`**

Next Lecture

- Introduction
  - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
  - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
  - Overview of Function Call
  - Memory Layout and Stack - x86-64 instructions and registers
  - Passing control
  - Passing data – Calling Conventions
  - Managing local data
  - Recursion
- Array
- Buffer Overflow
- Floating-point operations